

**CSA1718 AI**  
**LAB PROBLEMS**

**SANJAI.S(192511018)**

**BFS**

**AIM**

**To implement the Breadth First Search (BFS) algorithm to traverse all the nodes of a graph in breadth-first order.**

**ALGORITHM:**

- 1. Start from the source node.**
- 2. Mark the source node as visited.**
- 3. Insert the source node into a queue.**
- 4. While the queue is not empty:**
  - Remove the front node from the queue.**
  - Visit and display the node.**
  - For each unvisited adjacent node:**
    - Mark it as visited.**
    - Insert it into the queue.**
- 5. Repeat until the queue becomes empty.**

**PSEUDO CODE**

**START**

**Create an empty Queue**

**Create an empty Visited list**

**Mark Start node as Visited**

**Enqueue(Start)**

**WHILE Queue is not empty DO**

**Node  $\leftarrow$  Dequeue()**

**Print Node**

**FOR each adjacent node of Node DO**

**IF adjacent node is not Visited THEN**

**Mark it as Visited**

**Enqueue(adjacent node)**

**ENDIF**

**ENDFOR**

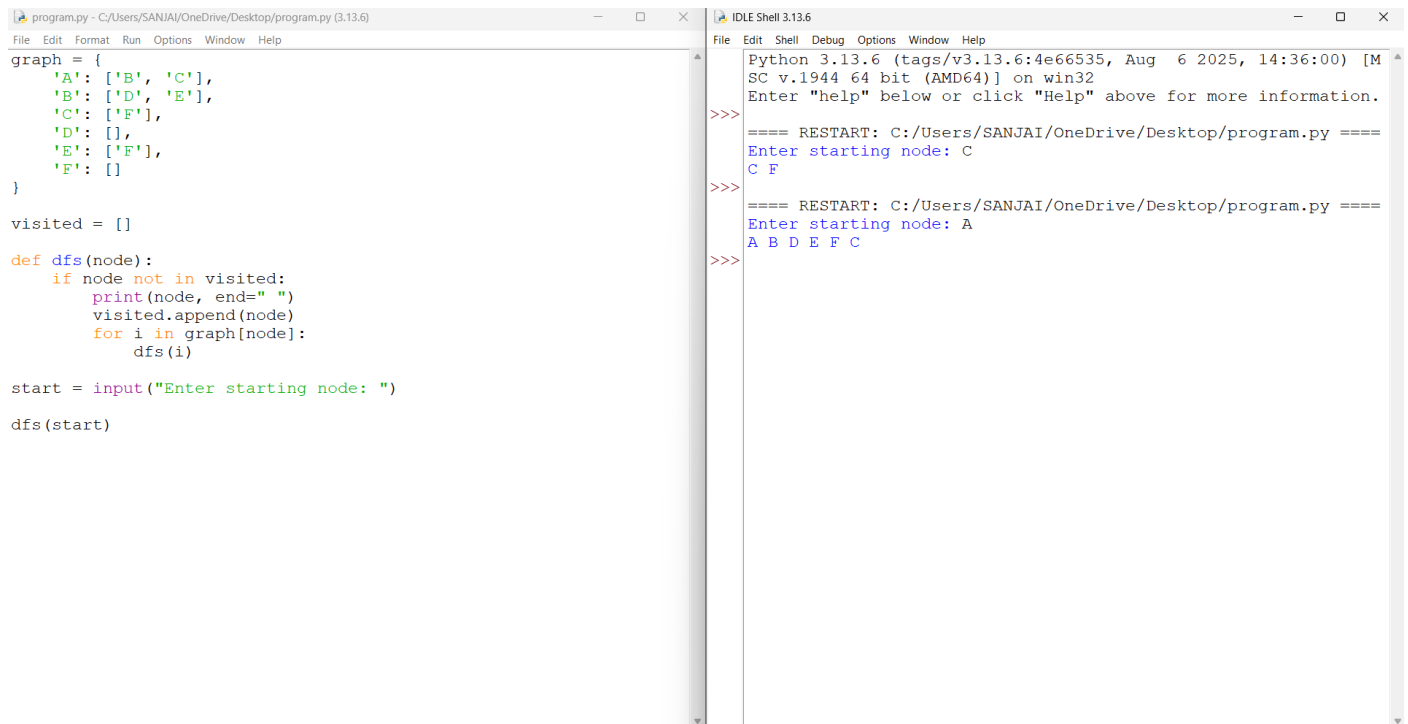
**ENDWHILE**

**STOP**

**Result:**

**The Breadth First Search (BFS) algorithm was successfully implemented, and all the nodes of the graph were traversed in breadth-first order using a queue.**

## OUTPUT:



The screenshot shows a Python IDE with two windows. The left window, titled 'program.py', contains the following code:

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

visited = []

def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.append(node)
        for i in graph[node]:
            dfs(i)

start = input("Enter starting node: ")
dfs(start)
```

The right window, titled 'IDLE Shell 3.13.6', shows the output of the program:

```
Python 3.13.6 (tags/v3.13.6:4e66535, Aug 6 2025, 14:36:00) [M
SC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/SANJAI/OneDrive/Desktop/program.py ====
Enter starting node: C
C F
>>>
==== RESTART: C:/Users/SANJAI/OneDrive/Desktop/program.py ====
Enter starting node: A
A B D E F C
>>>
```

## DFS

### AIM

To implement the Depth First Search (DFS) algorithm to traverse all the nodes of a graph in depth-first order.

### ALGORITHM

1. Start from the source node.
2. Mark the source node as visited.
3. Visit and display the node.
4. Recursively visit each unvisited adjacent node.
5. Repeat until all reachable nodes are visited.

### PSEUDO CODE

#### START

Create an empty Visited list

#### DFS(Node)

Mark Node as Visited

**Print Node**

**FOR each adjacent node of Node DO**

**IF adjacent node is not Visited THEN**

**DFS(adjacent node)**

**ENDIF**

**ENDFOR**

**END DFS**

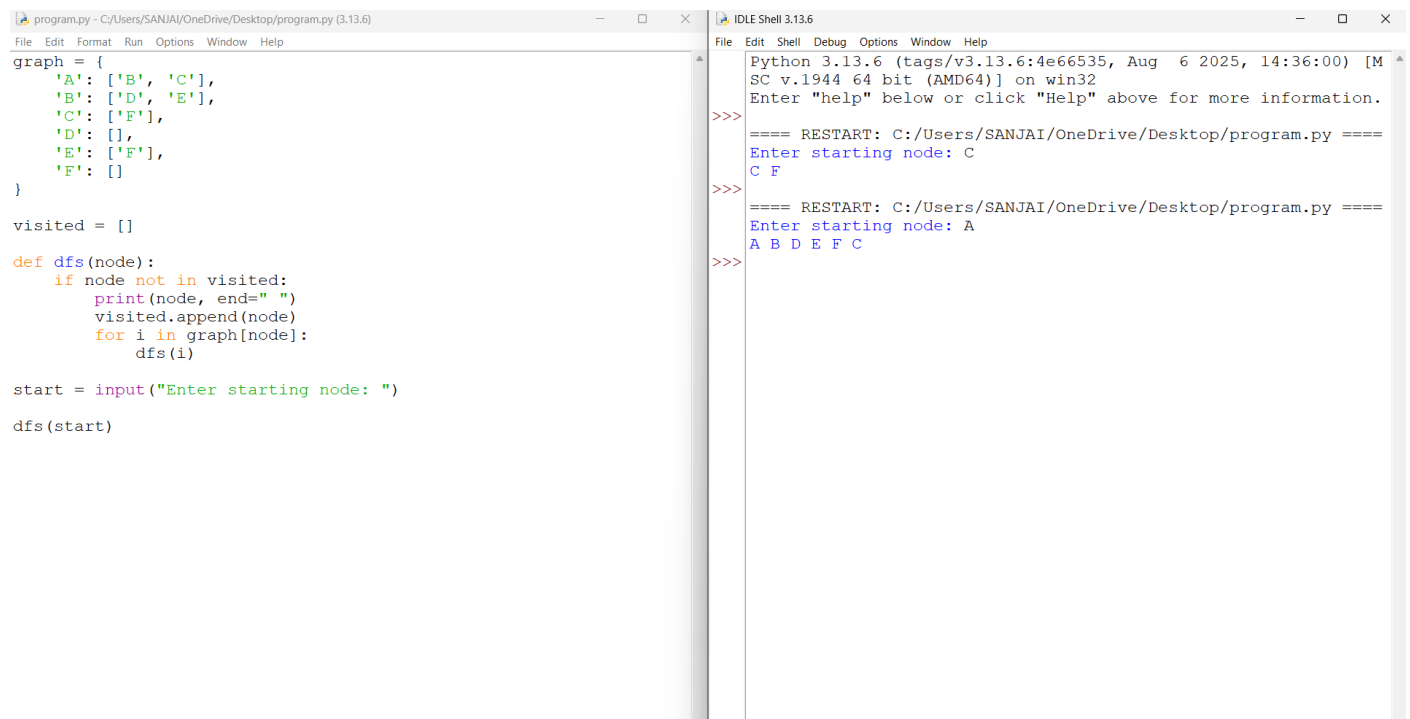
**Call DFS(Start Node)**

**STOP**

## RESULT

The Depth First Search (DFS) algorithm was successfully implemented, and all the nodes of the graph were traversed in depth-first order using recursion.

## OUTPUT:



The screenshot displays a Python IDE with two windows. The left window, titled 'program.py - C:/Users/SANJAI/OneDrive/Desktop/program.py (3.13.6)', contains the following code:

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

visited = []

def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.append(node)
        for i in graph[node]:
            dfs(i)

start = input("Enter starting node: ")
dfs(start)
```

The right window, titled 'IDLE Shell 3.13.6', shows the execution output:

```
Python 3.13.6 (tags/v3.13.6:4e66535, Aug 6 2025, 14:36:00) [M
SC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/SANJAI/OneDrive/Desktop/program.py ====
Enter starting node: C
C F
>>>
==== RESTART: C:/Users/SANJAI/OneDrive/Desktop/program.py ====
Enter starting node: A
A B D E F C
>>>
```